

Fuente de conocimiento extendida: Herencia en POO

Introducción

La herencia es un mecanismo que permite construir nuevas clases a partir de otras existentes. Es fundamental para la reutilización de código, la extensibilidad y la representación de jerarquías del mundo real. En desarrollo profesional se utiliza para modelar dominios complejos, compartir comportamiento común y aplicar principios SOLID.

Terminología

La clase padre define atributos y métodos comunes. La subclase reutiliza esos elementos y puede extenderlos. También puede sobrescribir comportamientos para especializarlos según el contexto del problema.

Relación con otros pilares

La herencia trabaja junto con encapsulamiento, abstracción y polimorfismo. Una buena jerarquía orientada a objetos usa encapsulamiento para proteger estado interno y polimorfismo para desacoplar comportamientos.

Ejemplo académico

Supongamos una universidad que modela personas. Todas comparten nombre, edad y método presentarse(). A partir de Persona se derivan Estudiante y Profesor.

```
class Persona {
    String nombre;
    int edad;

    public void presentarse() {
        System.out.println("Hola, soy " + nombre);
    }
}

class Estudiante extends Persona {
    String matricula;

    @Override
    public void presentarse() {
        System.out.println("Soy estudiante: " + nombre);
    }
}

class Profesor extends Persona {
    String especialidad;
```

```

@Override
public void presentarse() {
    System.out.println("Soy profesor de " + especialidad);
}
}

```

Tabla comparativa

Concepto	Descripción	Ejemplo
Superclase	Clase general con comportamiento común	Persona
Subclase	Clase especializada	Estudiante
Override	Redefinir método heredado	presentarse()
Polimorfismo	Misma interfaz, distinto comportamiento	Persona p = new Profesor()

Caso práctico profesional

En un sistema de ventas, Producto puede ser la superclase. De ella heredan ProductoDigital y ProductoFisico. Cada uno implementa calcularCostoEnvio() de forma distinta. Este ejemplo permite a tu sistema generar preguntas sobre análisis de código, modelado y aplicación en casos reales.

Buenas prácticas

- Usar herencia solo cuando exista una relación 'es-un'.
- Preferir composición cuando la relación sea 'tiene-un'.
- Evitar jerarquías profundas.
- Sobrescribir métodos con @Override.
- Combinar con clases abstractas e interfaces.

Preguntas esperadas

Este documento debe permitir generar preguntas conceptuales, interpretación de código, predicción de salida, diseño UML, refactorización y casos de arquitectura orientada a objetos.